

Summary: Assignment 2 — Stochastic Multi-Elevator Controller

A breakdown of what the assignment demanded and how **ex2.py** satisfies it.

1. What the assignment asked for

Based on the stub, engine (*ext_elev.py*), grader (*ex2_check.py*), and README, the demands are:

Requirement	Source
Implement Controller.choose_next_action(state) returning one legal action string per step	<i>ex2.py</i> stub
Return one of MOVE{e,f} , ENTER{p,e} , EXIT{p,e} , RESET	engine contract
State is (elevators_t , persons_t , total_persons_remaining)	engine
Interact only through GameAPI — touching the underlying Game = grade of zero	engine-access policy
Handle a stochastic MDP : elevator moves and person enter/exit can fail	<i>Game._apply_*</i>
Maximize reward across 33 problems (11 layouts × easy/medium/hard) × 30 seeds, within a step horizon	<i>ex2_check.py</i>
Exploit that delivering the last person grants goal_reward and resets the layout (rewards can be re-farmed)	engine + README
Match/beat baselines: random , sol1_h3/h5/h6 , sol2	<i>baseline/summary.md</i>

2. Overall approach

The implementation is a **greedy, priority-ranked reactive controller** (not a planner/MDP solver). Each call picks the single best legal action by walking down a fixed priority ladder, with scores that fold in **expected reward**, **action success probabilities**, and **distance**. This is a pragmatic choice for a stochastic setting where full planning is expensive and outcomes are noisy.

3. Initialization & precomputation (`__init__`)

All static data is front-loaded via the API and cached so per-step decisions are cheap:

- Pulls **get_reachable**, **get_capacities**, **get_goal_reward**, plus lazy caches for person goal/weight/reward-mean and the **stochastic probabilities** (*_person_prob*, *_elev_prob*).
- Builds an **elevator transfer graph**: *_shared_floors* (floors two elevators have in common) and an adjacency list *_adj*, enabling multi-elevator handoffs when no single elevator reaches a goal.
- Precomputes **reset-farming economics**: *_compute_loop_rps* finds the best reward-per-step single-person delivery loop, and a break-even threshold for when farming beats full delivery.

4. The decision ladder (`choose_next_action`)

The action is chosen by the first rule that fires:

Step	Rule	Description
1	EXIT at goal	Drop a rider who is already on their goal floor (collect reward).
2	EXIT for transfer	If the current elevator can't reach a rider's goal, drop them on a shared floor toward a routed next elevator.
3	ENTER	Board a waiting person if capacity allows; scored by $\text{reward_mean} \times \text{person_prob} \times \text{elev_prob}$, with a large bonus when the elevator can reach the goal directly, and a filter to avoid boarding the wrong elevator on a transfer chain.
4	MOVE loaded	Drive a loaded elevator toward the nearest goal (or transfer floor), scored by reward and distance.
5	Reset-farm	RESET when looping a cheap high-reward delivery beats delivering low-value remainders.
6	MOVE empty	Send an empty elevator toward the most valuable reachable waiting person.
7	RESET	Fallback.

5. How it meets the harder demands

- **Stochasticity:** success probabilities are multiplied into every ENTER/MOVE score, so unreliable ("broken") elevators on hard tiers are deprioritized when a reliable alternative exists; loop step-counts are inflated by $1/\text{elev_prob}$.
- **Multi-elevator layouts (e.g., `_m3`):** the BFS router (`_route_next_elevator`) + `_best_transfer_floor` enable passenger handoffs across elevators with disjoint reachable sets.
- **Reset-farming layouts (`_rl`):** dynamic comparison of $\text{farming_rps} \times \text{steps_remaining}$ vs. current full-delivery value lets it switch into "farm one lucrative person + RESET" mode — exactly what the `rl` problems reward.

6. Compliance notes

- ■ Goes exclusively through **GameAPI** (`self.game.*`) — respects the engine-access policy.
- ■ Emits only the four legal action formats.
- ■ `id = ["322535436"]` is set.
- The repo also carries the Assignment 1 solution (`ex1_322535436.py`, `search.py`, `utils.py`), which the controller doesn't import at runtime — the A2 controller is self-contained and greedy rather than reusing the A1 A* search.

7. One thing worth checking

The deterministic optimum from A1 is referenced by the grader as `optimal_a1` to scale horizons, but the A2 controller doesn't use the A1 planner. That's a valid design, but a hybrid (short-horizon planning for the transfer chains, greedy elsewhere) is the natural next step to push scores on the harder `sol2`-dominated layouts shown in `baseline/summary.md`.